

作业一：MinHash 与 LSH 相似性搜索实验

姓名：胡林森 学号：2023015509

python代码

```
1  """
2  作业一：MinHash 与 LSH 相似性搜索实验
3  """
4
5  import random
6  import math
7  import itertools
8  from collections import defaultdict
9  import matplotlib
10 matplotlib.use('Agg')
11 import matplotlib.pyplot as plt
12 import matplotlib.patches as mpatches
13 import numpy as np
14
15 plt.rcParams['font.family'] = 'SimHei'
16 plt.rcParams['axes.spines.top'] = False
17 plt.rcParams['axes.spines.right'] = False
18
19 # 文档中数据
20
21 documents = {
22     "D1": "data science uses scientific methods to analyze data",
23     "D2": "data science uses scientific techniques to analyze datasets",
24     "D3": "deep learning models are widely used in computer vision",
25     "D4": "computer vision often uses deep neural networks",
26     "D5": "big data systems include Hadoop and Spark",
27     "D6": "Spark is widely used for distributed data processing",
28 }
29
30 # 第 1 步: Shingling (k=3)
31
32 def get_shingles(text, k=3):
33     words = text.lower().split()
34     return set(tuple(words[i:i+k]) for i in range(len(words) - k + 1))
35
36 print("=" * 60)
37 print("第 1 步: Shingling (k=3) ")
38 print("=" * 60)
39
40 shingles = {}
41 for doc_id, text in documents.items():
42     shingles[doc_id] = get_shingles(text, k=3)
43     print(f"\n{doc_id}: {len(shingles[doc_id])} 个 shingles")
44     for s in sorted(shingles[doc_id]):
45         print(f"    {' '.join(s)}")
46
47 for da, db in [("D1", "D2"), ("D3", "D4")]:
48     inter = shingles[da] & shingles[db]
```

```

49     union = shingles[da] | shingles[db]
50     print(f"\n--- 选择文档对 {da} & {db} ---")
51     print(f"{da} ∩ {db} = {len(inter)} 个: {[ ' '.join(s) for s in
sorted(inter)]}")
52     print(f"{da} ∪ {db} = {len(union)} 个")
53
54 # 第 2 步: Jaccard 相似度
55
56 def jaccard(set_a, set_b):
57     inter = set_a & set_b
58     union = set_a | set_b
59     return len(inter) / len(union) if union else 0.0
60
61 print("\n" + "=" * 60)
62 print("第 2 步: Jaccard 相似度")
63 print("=" * 60)
64
65 doc_ids = list(documents.keys())
66 jaccard_scores = {}
67 for i in range(len(doc_ids)):
68     for j in range(i + 1, len(doc_ids)):
69         d1, d2 = doc_ids[i], doc_ids[j]
70         score = jaccard(shingles[d1], shingles[d2])
71         jaccard_scores[(d1, d2)] = score
72         print(f"J({d1}, {d2}) = {score:.4f}")
73
74 max_pair = max(jaccard_scores, key=jaccard_scores.get)
75 print(f"\n最相似文档对: {max_pair}, Jaccard = {jaccard_scores[max_pair]:.4f}")
76
77 # 第 3 步: MinHash (100 个 hash 函数)
78
79 print("\n" + "=" * 60)
80 print("第 3 步: MinHash (100 个 hash 函数)")
81 print("=" * 60)
82
83 all_shingles = set()
84 for s in shingles.values():
85     all_shingles |= s
86 shingle_list = sorted(all_shingles)
87 shingle_to_id = {s: i for i, s in enumerate(shingle_list)}
88
89 NUM_HASH = 100
90 PRIME = 2**31 - 1
91
92 random.seed(42)
93 hash_params = [(random.randint(1, PRIME - 1), random.randint(0, PRIME - 1))
94                 for _ in range(NUM_HASH)]
95
96 def minhash_signature(shingle_set, hash_params, prime):
97     ids = [shingle_to_id[s] for s in shingle_set]
98     sig = []
99     for a, b in hash_params:
100         min_val = min((a * x + b) % prime for x in ids)
101         sig.append(min_val)
102     return sig
103

```

```

104 signatures = {}
105 for doc_id in documents:
106     signatures[doc_id] = minhash_signature(shingles[doc_id], hash_params,
107     PRIME)
108     print(f"{doc_id} signature (前10): {signatures[doc_id][:10]}")
109
110 def minhash_similarity(sig_a, sig_b):
111     return sum(1 for a, b in zip(sig_a, sig_b) if a == b) / len(sig_a)
112
113 minhash_scores = {}
114 print("\n--- MinHash vs Jaccard 对比 ---")
115 print(f"{'文档对':<15} {'Jaccard':>10} {'MinHash':>10} {'差值':>10}")
116 print("-" * 44)
117 for (d1, d2), j_score in jaccard_scores.items():
118     m_score = minhash_similarity(signatures[d1], signatures[d2])
119     minhash_scores[(d1, d2)] = m_score
120     error = abs(j_score - m_score)
121     print(f"({d1},{d2})          {j_score:>10.4f} {m_score:>10.4f}
122     {error:>10.4f}")
123
124 # 第 4 步: LSH (bands=50, rows=2)
125 BANDS = 50
126 ROWS = 2
127 assert BANDS * ROWS == NUM_HASH
128
129 print("\n" + "=" * 60)
130 print(f"第 4 步: LSH (bands={BANDS}, rows={ROWS}) ")
131 print("=" * 60)
132 threshold_approx = (1 / BANDS) ** (1 / ROWS)
133 print(f" (b={BANDS}, r={ROWS} 阈值约为 {threshold_approx:.2f}) ")
134
135 def lsh_candidate_pairs(signatures, b, r):
136     buckets = defaultdict(set)
137     for doc_id, sig in signatures.items():
138         for band_idx in range(b):
139             band = tuple(sig[band_idx * r: (band_idx + 1) * r])
140             buckets[(band_idx, band)].add(doc_id)
141     candidates = set()
142     for bucket_docs in buckets.values():
143         if len(bucket_docs) > 1:
144             docs = sorted(bucket_docs)
145             for i in range(len(docs)):
146                 for j in range(i + 1, len(docs)):
147                     candidates.add((docs[i], docs[j]))
148     return candidates
149
150 candidate_pairs = lsh_candidate_pairs(signatures, BANDS, ROWS)
151 print(f"LSH 找到的 candidate pairs (共 {len(candidate_pairs)} 对): ")
152 for pair in sorted(candidate_pairs):
153     j = jaccard(shingles[pair[0]], shingles[pair[1]])
154     print(f" {pair} → Jaccard = {j:.4f}")
155
156 high_sim_pairs = [p for p, s in jaccard_scores.items() if s >= 0.1]
157 print(f"\nJaccard >= 0.1 的文档对: {high_sim_pairs}")
158 print(f"LSH candidate pairs: {sorted(candidate_pairs)}")

```

```

158 hits = [p for p in high_sim_pairs if p in candidate_pairs]
159 misses = [p for p in high_sim_pairs if p not in candidate_pairs]
160 false_pos = [p for p in candidate_pairs if p not in high_sim_pairs]
161 print(f"\nLSH 命中高相似度对: {hits}")
162 print(f"LSH 遗漏 (false negative): {misses}")
163 print(f"LSH 误报 (false positive): {false_pos}")
164
165 # S 曲线分析
166 print("\n" + "=" * 60)
167 print("LSH 概率 (S 曲线) 分析")
168 print("=" * 60)
169
170 configs = [(20, 5), (10, 10), (50, 2)]
171 print(f"{'s':>6} {'P(b=20,r=5)':>14} {'P(b=10,r=10)':>14} {'P(b=50,r=2)':>14}")
172 print("-" * 52)
173 for s in [0.1, 0.2, 0.3, 0.5, 0.7, 0.8, 0.9, 1.0]:
174     row = f"{'s':>6.1f}"
175     for b_, r_ in configs:
176         prob = 1 - (1 - s**r_)**b_
177         row += f" {'prob:>14.4f}"
178     print(row)
179
180
181
182 # 可视化
183 BLUE = '#378ADD'
184 PINK = '#D4537E'
185 GREEN = '#1D9E75'
186 AMBER = '#BA7517'
187 GRAY = '#888780'
188 LGRAY = '#E8E6E0'
189
190 fig = plt.figure(figsize=(14, 11))
191 fig.patch.set_facecolor('white')
192 print("=" * 60)
193 print("实验完成! 开始生成可视化图表...")
194 print("=" * 60)
195
196 # — LSH S 曲线 —————
197 s_arr = np.linspace(0, 1, 200)
198 curve_configs = [
199     (20, 5, BLUE, 'b=20, r=5'),
200     (10, 10, PINK, 'b=10, r=10'),
201     (50, 2, GREEN, 'b=50, r=2'),
202 ]
203
204 fig = plt.figure(figsize=(7, 7))
205 fig.patch.set_facecolor('white')
206
207 ax4 = fig.add_subplot()
208
209 for b_, r_, color, label in curve_configs:
210     probs = 1 - (1 - s_arr**r_)**b_
211     t = (1/b_)**(1/r_)
212     ax4.plot(s_arr, probs, color=color, linewidth=2, label=label)

```

```

213     ax4.axvline(t, color=color, linewidth=0.8, linestyle='--', alpha=0.5)
214
215     # 标注实验中 D1-D2 的 Jaccard
216     j12 = jaccard_scores[('D1', 'D2')]
217     ax4.axvline(j12, color=AMBER, linewidth=1.5, linestyle=':',
218                 label=f'J(D1,D2)={j12:.2f}')
219
220     ax4.fill_betweenx([0, 1], 0, j12, alpha=0.06, color=AMBER)
221
222     ax4.set_xlim(0, 1); ax4.set_ylim(0, 1.05)
223     ax4.set_xlabel('Jaccard相似度 s', fontsize=9, color=GRAY)
224     ax4.set_ylabel('候选对概率 P', fontsize=9, color=GRAY)
225     ax4.set_title('LSH S 曲线', fontsize=11, pad=10, color='#2C2C2A')
226     ax4.tick_params(colors=GRAY)
227     ax4.legend(fontsize=8.5, frameon=False, loc='upper left')
228     ax4.text(0.02, 0.55, '理想\n阶跃', fontsize=7, color=GRAY, alpha=0.6)
229
230     # 理想阶跃虚线（参考）
231     ax4.step([0, 0.5, 0.5, 1], [0, 0, 1, 1], color=GRAY, linewidth=0.8,
232             linestyle=(0, (3, 5)), alpha=0.4)
233
234     plt.savefig('minhash_lsh_visualization.png',
235               dpi=150, bbox_inches='tight', facecolor='white')
236     print("\n图表已保存: minhash_lsh_visualization.png")
237     plt.close()

```

输出结果

截图

PS E:\CUPK-lib\大数据分析算法> & C:/Users/15638/anaconda3/python.exe e:/CUPK-lib/大数据分析算法/1.MinHash与LSH相似性搜索实验/01.py

=====

Shingling (k=3)

=====

D1: 6 个 shingles

data science uses
methods to analyze
science uses scientific
scientific methods to
to analyze data
uses scientific methods

D2: 6 个 shingles

data science uses
science uses scientific
scientific techniques to
techniques to analyze
to analyze datasets
uses scientific techniques

D3: 7 个 shingles

are widely used
deep learning models
in computer vision
learning models are
models are widely
used in computer
widely used in

D4: 5 个 shingles

computer vision often
deep neural networks
often uses deep
uses deep neural
vision often uses

D5: 5 个 shingles

big data systems
data systems include
hadoop and spark
include hadoop and
systems include hadoop

D6: 6 个 shingles

distributed data processing
for distributed data
is widely used
spark is widely
used for distributed
widely used for

--- 选择文档对 D1 & D2 ---

D1 n D2 = 2 个: ['data science uses', 'science uses scientific']

D1 $\cup$ D2 = 10 个

--- 选择文档对 D3 & D4 ---

D3 n D4 = 0 个: []

D3 $\cup$ D4 = 12 个

=====

Jaccard 相似度

=====

J(D1, D2) = 0.2000

J(D1, D3) = 0.0000

J(D1, D4) = 0.0000

J(D1, D5) = 0.0000

J(D1, D6) = 0.0000

J(D2, D3) = 0.0000

J(D2, D4) = 0.0000

J(D2, D5) = 0.0000

J(D2, D6) = 0.0000

J(D3, D4) = 0.0000

J(D3, D5) = 0.0000

J(D3, D6) = 0.0000

J(D4, D5) = 0.0000

最相似文档对: ('D1', 'D2'), Jaccard = 0.2000

=====

MinHash (100 个 hash 函数)

=====

D1 signature (前10): [32817013, 196926524, 150280525, 337866456, 88762247, 460550501, 77052773, 53214063, 439061021, 393176149]

D2 signature (前10): [63590190, 358057079, 150280525, 107748505, 204973616, 525186444, 372195508, 147000856, 103717816, 393176149]

D3 signature (前10): [239081663, 143216339, 98419537, 88642983, 220106707, 266642672, 13452652, 54087935, 3758422, 101374894]

D4 signature (前10): [631650580, 304346894, 1064661240, 356971978, 422263263, 24811705, 667338243, 240787649, 909828643, 329187101]

D5 signature (前10): [311440811, 35795969, 44039786, 69537461, 103895338, 395914558, 244995266, 800886791, 974323014, 265198053]

D6 signature (前10): [208308486, 89506154, 259040027, 126854027, 553607723, 331278615, 435795629, 427487363, 203677210, 37385846]

--- MinHash vs Jaccard 对比 ---

文档对	Jaccard	MinHash	差值
-----	---------	---------	----

(D1,D2)	0.2000	0.2800	0.0800
---------	--------	--------	--------

(D1,D3)	0.0000	0.0000	0.0000
---------	--------	--------	--------

(D1,D4)	0.0000	0.0000	0.0000
---------	--------	--------	--------

(D1,D5)	0.0000	0.0000	0.0000
---------	--------	--------	--------

(D1,D6)	0.0000	0.0000	0.0000
---------	--------	--------	--------

(D2,D3)	0.0000	0.0000	0.0000
---------	--------	--------	--------

(D2,D4)	0.0000	0.0000	0.0000
---------	--------	--------	--------

(D2,D5)	0.0000	0.0000	0.0000
---------	--------	--------	--------

(D2,D6)	0.0000	0.0000	0.0000
---------	--------	--------	--------

(D3,D4)	0.0000	0.0000	0.0000
---------	--------	--------	--------

(D3,D5)	0.0000	0.0000	0.0000
---------	--------	--------	--------

(D3,D6)	0.0000	0.0000	0.0000
---------	--------	--------	--------

(D4,D5)	0.0000	0.0000	0.0000
---------	--------	--------	--------

(D4,D6)	0.0000	0.0000	0.0000
---------	--------	--------	--------

(D5,D6)	0.0000	0.0000	0.0000
---------	--------	--------	--------

```
=====
(b=50, r=2 阈值约为 0.14)
LSH 找到的 candidate pairs (共 1 对):
('D1', 'D2') → Jaccard = 0.2000

Jaccard >= 0.1 的文档对: [('D1', 'D2')]
LSH candidate pairs: [('D1', 'D2')]

LSH 命中高相似度对: [('D1', 'D2')]
LSH 遗漏 (false negative): []
LSH 误报 (false positive): []

=====
LSH 概率 (s 曲线) 分析
=====
s      P(b=20,r=5)  P(b=10,r=10)  P(b=50,r=2)
-----
0.1      0.0002      0.0000      0.3950
0.2      0.0064      0.0000      0.8701
0.3      0.0475      0.0001      0.9910
0.5      0.4701      0.0097      1.0000
0.7      0.9748      0.2491      1.0000
0.8      0.9996      0.6789      1.0000
0.9      1.0000      0.9863      1.0000
1.0      1.0000      1.0000      1.0000

图表已保存: minhash_lsh_visualization.png
```

Shingling 结果 (k=3)

- D1: 6 个 shingles
- data science uses
 - methods to analyze
 - science uses scientific
 - scientific methods to
 - to analyze data
 - uses scientific methods

- D2: 6 个 shingles
- data science uses
 - science uses scientific
 - scientific techniques to
 - techniques to analyze
 - to analyze datasets
 - uses scientific techniques

- D3: 7 个 shingles
- are widely used
 - deep learning models
 - in computer vision
 - learning models are
 - models are widely
 - used in computer
 - widely used in

- D4: 5 个 shingles
- computer vision often
 - deep neural networks
 - often uses deep
 - uses deep neural
 - vision often uses

- D5: 5 个 shingles
- big data systems
 - data systems include
 - hadoop and spark

include hadoop and
systems include hadoop

D6: 6 个 shingles
distributed data processing
for distributed data
is widely used
spark is widely
used for distributed
widely used for

--- 选择文档对 D1 & D2 ---

$D1 \cap D2 = 2$ 个: ['data science uses', 'science uses scientific']

$D1 \cup D2 = 10$ 个

--- 选择文档对 D3 & D4 ---

$D3 \cap D4 = 0$ 个: []

$D3 \cup D4 = 12$ 个

Jaccard 相似度结果

$J(D1, D2) = 0.2000$
 $J(D1, D3) = 0.0000$
 $J(D1, D4) = 0.0000$
 $J(D1, D5) = 0.0000$
 $J(D1, D6) = 0.0000$
 $J(D2, D3) = 0.0000$
 $J(D2, D4) = 0.0000$
 $J(D2, D5) = 0.0000$
 $J(D2, D6) = 0.0000$
 $J(D3, D4) = 0.0000$
 $J(D3, D5) = 0.0000$
 $J(D3, D6) = 0.0000$
 $J(D4, D5) = 0.0000$
 $J(D4, D6) = 0.0000$
 $J(D5, D6) = 0.0000$

最相似文档对: ('D1', 'D2'), Jaccard = 0.2000

MinHash 相似度结果

D1 signature (前10) : [32817013, 196926524, 150280525, 337866456, 88762247, 460550501, 77052773, 53214063, 439061021, 393176149]
D2 signature (前10) : [63590190, 358057079, 150280525, 107748505, 204973616, 525186444, 372195508, 147000856, 103717816, 393176149]
D3 signature (前10) : [239081663, 143216339, 98419537, 88642983, 220106707, 266642672, 13452652, 54087935, 3758422, 101374894]
D4 signature (前10) : [631650580, 304346894, 1064661240, 356971978, 422263263, 24811705, 667338243, 240787649, 909828643, 329187101]
D5 signature (前10) : [311440811, 35795969, 44039786, 69537461, 103895338, 395914558, 244995266, 800886791, 974323014, 265198053]
D6 signature (前10) : [208308486, 89506154, 259040027, 126854027, 553607723, 331278615, 435795629, 427487363, 203677210, 37385846]

MinHash VS. Jaccard

文档对	Jaccard	MinHash	差值
(D1,D2)	0.2000	0.2800	0.0800
(D1,D3)	0.0000	0.0000	0.0000
(D1,D4)	0.0000	0.0000	0.0000
(D1,D5)	0.0000	0.0000	0.0000
(D1,D6)	0.0000	0.0000	0.0000
(D2,D3)	0.0000	0.0000	0.0000
(D2,D4)	0.0000	0.0000	0.0000
(D2,D5)	0.0000	0.0000	0.0000
(D2,D6)	0.0000	0.0000	0.0000
(D3,D4)	0.0000	0.0000	0.0000
(D3,D5)	0.0000	0.0000	0.0000
(D3,D6)	0.0000	0.0000	0.0000
(D4,D5)	0.0000	0.0000	0.0000
(D4,D6)	0.0000	0.0000	0.0000
(D5,D6)	0.0000	0.0000	0.0000

LSH candidate 结果

(b=50, r=2 阈值约为 0.14, 适合检测 D1-D2 的 Jaccard=0.2)

LSH 找到的 candidate pairs (共 1 对) :

('D1', 'D2') → Jaccard = 0.2000

Jaccard >= 0.1 的文档对: [('D1', 'D2')]

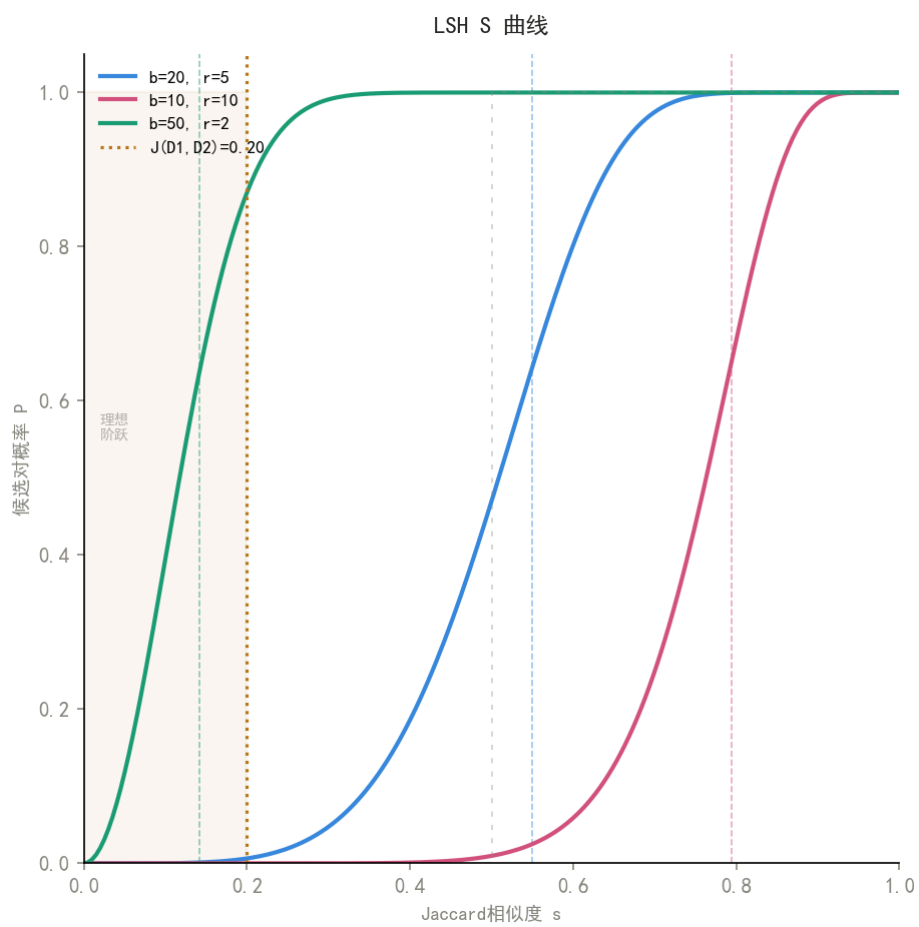
LSH candidate pairs: [('D1', 'D2')]

LSH 命中高相似度对: [('D1', 'D2')]

LSH 遗漏 (false negative) : []

LSH 误报 (false positive) : []

LSH 概率 (S 曲线) 分析



横轴是真实的 Jaccard 相似度 s , 纵轴是 LSH 把这对文档识别为 candidate pair 的概率 $P = 1 - (1 - s^r)^b$

s	P(b=20,r=5)	P(b=10,r=10)	P(b=50,r=2)
0.1	0.0002	0.0000	0.3950
0.2	0.0064	0.0000	0.8701
0.3	0.0475	0.0001	0.9910
0.5	0.4701	0.0097	1.0000
0.7	0.9748	0.2491	1.0000
0.8	0.9996	0.6789	1.0000
0.9	1.0000	0.9863	1.0000
1.0	1.0000	1.0000	1.0000

小结

配置	特点
b=20, r=5	中等陡峭， 阈值≈0.55
b=10, r=10	非常陡峭， 阈值≈0.79， 高精度但低召回
b=50, r=2	非常平缓， 阈值≈0.14， 高召回但误报多

总结

1. Jaccard 相似度计算方法

将文档转化为 shingle 集合，相似度 = 交集大小 / 并集大小。本实验中 D1 与 D2 共享 2 个 shingle， $J(D1,D2) = 2/10 = 0.2$ ，其余文档对词汇完全不重叠，相似度为 0。
Jaccard 直观准确，但需构建完整集合，在大规模数据下计算代价高。

2. MinHash 近似 Jaccard

对两个集合施加同一随机排列，取最小元素相同的概率恰好等于 Jaccard 相似度。用 100 个独立 hash 函数重复这一过程并取均值，得到 $MinHash(D1,D2) \approx 0.28$ ，与真实值 0.2 误差仅 0.08。

3. LSH 加速相似搜索

LSH 将长度为 100 的 signature 分成 50 个 band，每 band 2 行，只有某个 band 完全相同的文档对才成为 candidate pair，从而将候选集从 $O(n^2)$ 压缩到远小于 n^2 的规模。本实验中 LSH 精确找到唯一高相似对 (D1, D2)，无漏检无误报，而无需计算全部 15 对文档的相似度。

4. LSH 不一定找到最相似对

LSH 是概率算法，存在 false negative（高相似却未被选中）和 false positive（低相似却被选中）的风险。b 增大时阈值降低、召回率升高但误报增多，S 曲线左移；r 增大时阈值升高、精度提升但召回率下降，S 曲线右移且更陡峭。实际应用中需根据场景调整 b 与 r 的比例来平衡精度与召回。

5. Approximate Similarity Search 思想

不追求精确答案，而是用概率快速筛选出"很可能相似"的候选集。MinHash 用随机采样近似集合相似度，LSH 用分桶跳过大量不相似对，两者结合将复杂度从 $O(n^2)$ 压缩到近似 $O(n)$ ，在精度损失极小的前提下实现大规模数据的高效搜索，是大数据场景下近似算法优于暴力精确算法的典型体现。